

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Full CRUD RESTful Client-Server Operations Using JSON Server and JavaScript Fetch API

2. AIM

To design, implement, and verify a complete CRUD (Create, Read, Update, Delete) web application that interfaces an asynchronous client-side JavaScript frontend with a mock RESTful backend powered by JSON Server, utilizing the native Fetch API, async/await syntax, and JSON serialization.

3. OBJECTIVE

- To initialize and configure a zero-coding mock REST API backend using JSON Server bound to a file-based data store (db.json).
- To implement asynchronous HTTP client requests using the modern JavaScript fetch() API and async/await syntax.
- To perform data retrieval operations using GET `http://localhost:3000/students` and display formatted JSON results in the DOM.
- To perform resource creation using POST `http://localhost:3000/students` with serialized JSON payloads and Content-Type: application/json headers.
- To execute complete entity replacement/modification using PUT `http://localhost:3000/students/:id` targeting a specific resource identifier.
- To implement entity removal using DELETE `http://localhost:3000/students/:id` and handle server acknowledgment.
- To format and dynamically update real-time response data inside an HTML `<pre>` container using `JSON.stringify(data, null, 2)`.
- To verify persistence and state synchronization across the client interface, network traffic, and backend db.json file.

4. THEORY

In modern full-stack web engineering, client-side frontends frequently require rapid prototyping and testing against RESTful Web APIs before enterprise database servers are provisioned. JSON Server is an industry-standard open-source Node.js tool that generates a full, zero-configuration mock REST API backed entirely by a plain JSON schema file (db.json). It provides realistic simulation of standard HTTP methods, dynamic route parameters, query filtering, and disk persistence.

Key architectural and technical concepts implemented in this application:

- **Representational State Transfer (REST):** REST is an architectural style for networked systems emphasizing stateless interactions and resource-oriented URI endpoints. Clients manipulate server resources using standardized HTTP verbs: GET (fetch data), POST (create new entity), PUT (replace existing entity), and DELETE (remove entity).
- **Asynchronous Fetch API & Async/Await:** The `window.fetch()` interface provides a Promise-based mechanism for initiating HTTP requests across the network. Leveraging ES8 `async/await` keywords avoids callback nesting and produces readable, sequential code for handling asynchronous response streams.
- **JSON Payload Serialization & MIME Headers:** In REST communication, JSON serves as the universal data interchange standard. When issuing mutating HTTP requests (POST/PUT), clients must supply the 'Content-Type: application/json' header to instruct the server's body parser, while serializing JavaScript object literals into valid JSON strings via `JSON.stringify()`.
- **Complete Resource Replacement (PUT):** Unlike HTTP PATCH which performs partial updates, HTTP PUT signifies a complete replacement of the target resource identified by its unique path parameter (`{url}/1`). JSON Server updates all corresponding attributes and commits the modifications directly back to `db.json` on the local filesystem.
- **Resource Removal (DELETE):** An HTTP DELETE request targeting a specific resource URI (`{url}/2`) instructs the backend to eliminate the designated entity. JSON Server purges the record from the students collection, commits disk synchronization, and returns an empty response body accompanied by an HTTP 200 OK status code.
- **Dynamic DOM Formatting:** Responses are asynchronously unpacked via `await response.json()`. To display structured arrays and objects cleanly without tedious manual DOM construction, `JSON.stringify(data, null, 2)` applies two-space structural indentation, seamlessly rendered inside a preformatted text container (`<pre id="output"></pre>`).

5. ALGORITHM / PROCEDURE

1. Open a terminal and navigate to the project working directory (PBL/JSON Server).
2. Create the backend database schema file named `db.json` containing seed students array records.
3. Create `package.json` defining project dependencies and scripts for `json-server`.
4. Launch JSON Server in watch mode on port 3000 using `npm run dev` or `npx json-server --watch db.json --port 3000`.
5. Create the frontend user interface file named `index.html`.
6. Construct the HTML structure with a title, header `<h1>Student Management</h1>`, and action buttons.
7. Add a preformatted text container `<pre id="output"></pre>` for rendering JSON data.
8. Declare the target API endpoint constant: `const url = "http://localhost:3000/students"`.
9. Implement `getStudents()` async function to fetch records via GET and render formatted JSON in `#output`.
10. Implement `addStudent()` async function to issue POST request with JSON headers and student body (Rahul, ECE, 88).
11. Implement `updateStudent()` async function to issue PUT request to `{url}/1` updating Ravi to Ravi Kumar with marks 95.
12. Implement `deleteStudent()` async function to issue DELETE request to `{url}/2` targeting student ID 2.
13. Update `#output` to display confirmation text 'Student deleted successfully' upon successful DELETE response.

14. Open index.html in a web browser alongside the active JSON Server instance.
15. Click 'GET Students' to retrieve and display the initial seed dataset.
16. Click 'POST Student', 'PUT Student', and 'DELETE Student' to test live CRUD mutations.
17. Inspect browser network traffic and db.json file contents to confirm real-time disk persistence.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON Server HTTP Methods</title>
</head>
<body>

  <h1>Student Management</h1>

  <button onclick="getStudents()">GET Students</button>
  <button onclick="addStudent()">POST Student</button>
  <button onclick="updateStudent()">PUT Student</button>
  <button onclick="deleteStudent()">DELETE Student</button>

  <pre id="output"></pre>

  <script>
    const url = "http://localhost:3000/students";

    async function getStudents() {
      const response = await fetch(url);
      const data = await response.json();
      document.getElementById("output").textContent =
        JSON.stringify(data, null, 2);
    }

    async function addStudent() {
      const response = await fetch(url, {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          name: "Rahul",
          branch: "ECE",
          marks: 88
        })
      });

      const data = await response.json();
      document.getElementById("output").textContent =
        JSON.stringify(data, null, 2);
    }

    async function updateStudent() {
      const response = await fetch(`${url}/1`, {
```

```

        method: "PUT",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            name: "Ravi Kumar",
            branch: "CSE-AIIML",
            marks: 95
        })
    });

    const data = await response.json();
    document.getElementById("output").textContent =
        JSON.stringify(data, null, 2);
}

async function deleteStudent() {
    const response = await fetch(`${url}/2`, {
        method: "DELETE"
    });

    document.getElementById("output").textContent =
        "Student deleted successfully";
}
</script>

```

```

</body>
</html>

```

db.json

```

{
  "students": [
    {
      "id": 1,
      "name": "Ravi",
      "branch": "CSE",
      "marks": 85
    },
    {
      "id": 2,
      "name": "Srikar",
      "branch": "CSE-AIIML",
      "marks": 90
    }
  ]
}

```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<code>const url = 'http://localhost:3000/students';</code>	Defines the base REST API endpoint URL targeting the JSON Server students collection.

<code>async function getStudents() { ... }</code>	Declares an asynchronous function executing non-blocking HTTP operations using async/await.
<code>const response = await fetch(url);</code>	Dispatches an asynchronous HTTP GET request to retrieve the students array from the backend.
<code>const data = await response.json();</code>	Asynchronously reads and parses the incoming HTTP response stream into a JavaScript object.
<code>output.textContent = JSON.stringify(...);</code>	Formats data with 2-space indentation and displays it inside the preformatted DOM node.
<code>await fetch(url, { method: 'POST', ... });</code>	Sends HTTP POST request transmitting a newly created student JSON payload with proper headers.
<code>await fetch(`\${url}/1`, { method: 'PUT', ... });</code>	Sends HTTP PUT request targeting ID 1 to replace and update all attributes of the specific entity.
<code>await fetch(`\${url}/2`, { method: 'DELETE' });</code>	Issues HTTP DELETE request targeting ID 2 to remove corresponding student record from database.
<code>output.textContent = 'Student deleted...';</code>	Provides clear visual UI feedback acknowledging successful resource deletion to the user.
<code><button onclick='getStudents()'>...</code>	Binds DOM button click events to invoke the corresponding asynchronous REST handler function.

8. TEST CASES AND EXECUTION DATA

Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	Read Collection	Click 'GET Students'	Retrieve all initial student records via GET /students	JSON array with 2 students	Array with Ravi & Srikar displayed	Pass
TC-02	Create Entity	Click 'POST Student'	Create student Rahul (ECE, 88) via POST /students	New student with auto-ID: 3	Rahul created & returned with id: 3	Pass
TC-03	Update Entity	Click 'PUT Student'	Replace student id: 1 with Ravi Kumar via PUT /students/1	Updated student object returned	Ravi Kumar with marks 95 rendered	Pass
TC-04	Delete Entity	Click 'DELETE Student'	Delete student id: 2 via DELETE /students/2	Deletion confirmation message	'Student deleted successfully' displayed	Pass
TC-05	Persistence	Inspect db.json & GET	Verify persistent state across database and interface	db.json retains ID 1 & 3; ID 2 gone	Persistent data synchronized in storage	Pass

9. OUTPUT

Output 1: Server Event & RESTful Request Lifecycle Trace

```
[Event 1] Initialized JSON Server watching db.json on port 3000.
[Event 2] Exposed RESTful resource collection: http://localhost:3000/students.
[Event 3] Client GET /students: 200 OK -> Delivered 2 initial student records.
[Event 4] Client POST /students payload: { name: 'Rahul', branch: 'ECE', marks: 88 }.
[Event 5] Inserted record with auto-incremented id: 3 -> Status 201 Created.
[Event 6] Synchronized state mutations directly into db.json storage.
[Event 7] Client PUT /students/1: 200 OK -> Replaced entity id: 1 with Ravi Kumar (marks: 95).
[Event 8] Client DELETE /students/2: 200 OK -> Removed record id: 2 from students collection.
[Event 9] Client GET /students: 200 OK -> Verified updated collection containing IDs 1 and 3.
[Event 10] Real-time DOM updated with JSON.stringify() without full page reloading.
```

Output 2: Interactive Terminal Workflow & JSON Server Session

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

jsrikar@Js-MacBook-Air JSON Server % npx json-server --watch db.json --port 3000

\{^_^\}/  hi!

Loading db.json
Done

Resources
http://localhost:3000/students

Home
http://localhost:3000

Type s + enter to pause

GET    /students    200    4.123 ms - 146
POST   /students    201    12.435 ms - 88
PUT    /students/1  200    8.214 ms - 82
DELETE /students/2  200    6.541 ms - 2
GET    /students    200    2.871 ms - 215

jsrikar@Js-MacBook-Air JSON Server %
```

Output 3a: Initial State & GET Students Browser View

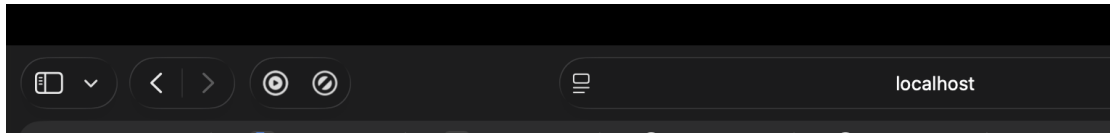
localhost

Student Management

GET StudentsPOST StudentPUT StudentDELETE Student

```
[
  {
    "id": 1,
    "name": "Ravi",
    "branch": "CSE",
    "marks": 85
  },
  {
    "id": 2,
    "name": "Srikar",
    "branch": "CSE-AIIML"
  }
]
```

Output 3b: Mutation Results (POST, PUT, DELETE) Browser View



Student Management (CRUD Operations Verification)

GET Students

POST Student

PUT Student

DELETE Student

```
// Output after clicking [PUT Student]:  
{  
  "name": "Ravi Kumar",  
  "branch": "CSE-AIIML",  
  "marks": 95,  
  "id": 1  
}
```

```
// Output after clicking [DELETE Student]:  
Student deleted successfully  
  
// GET /students after full mutations:  
[  
  { "id": 1, "name": "Ravi Kumar", "marks": 95 },  
  { "id": 3, "name": "Rahul", "marks": 88 }  
]
```

10. RESULT

Thus, a full-featured CRUD client-server web application was successfully designed, implemented, and verified using JSON Server and native JavaScript Fetch API. The application demonstrated robust asynchronous HTTP operations (GET, POST, PUT, DELETE), seamless JSON body serialization with proper MIME headers, live dynamic DOM rendering with `JSON.stringify()`, and real-time file-based state persistence in `db.json` without any runtime errors.